Microbiology LIMS POC — Process Reference (with Block Diagrams)

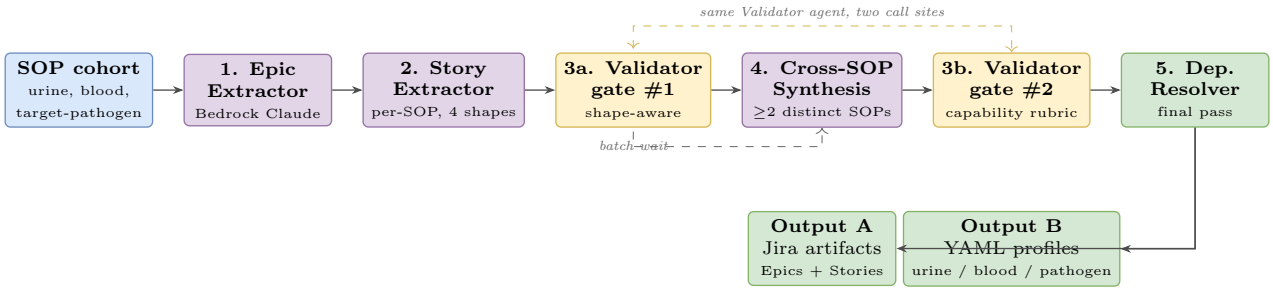
Visual companion to `microbio_lims_architecture.drawio` | 2026-04-29

What the system does

An SME drops microbiology SOPs into S3. An agent pipeline, grounded in a hybrid retrieval substrate (BM25 + dense), reads them and emits two artifacts: **(A)** Jira-ready Epic/Story tree and **(B)** per-culture YAML configuration profiles (urine, blood, target-pathogen). The Cyto Jira backlog is a *teaching corpus* (few-shot exemplars), not a registry. The agent stops at Story — Tasks are dev-authored.

Color legend: src source/data cmp compute ai AI/LLM agent val validator/decision store storage/output xc cross-cutting

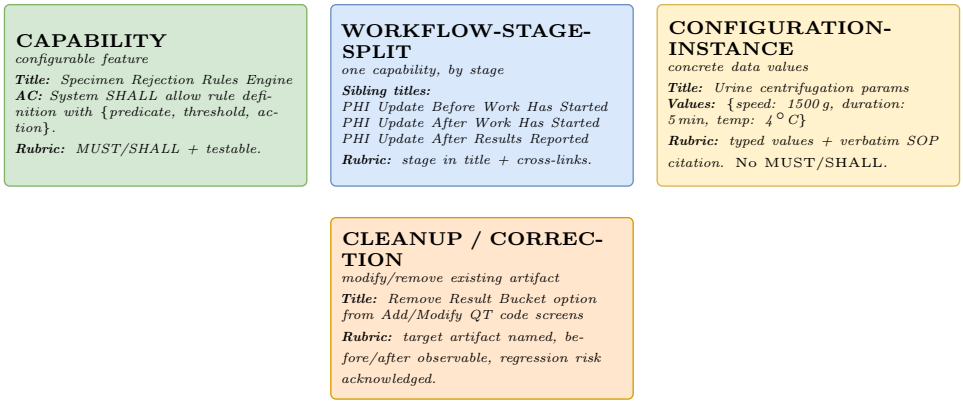
Agent pipeline — 5 agents, Validator gates twice



What each box does and emits:

1. **Epic Extractor** — identifies capability domains. *Out:* {`id, title, discipline, themes[], cyto_epic_analog?`}.
2. **Story Extractor** — per-SOP, multi-shape. Each Story declares its `shape` for downstream routing.
3. **Validator gate #1** — shape verification + shape-specific sub-rubric. Two revisions then SME-escalate.
4. **Cross-SOP Synthesis** — runs after gate #1 clears the entire batch. Behavioral-similarity clustering over (title + AC + source excerpt); a cluster qualifies only if its members come from ≥ 2 *distinct* SOPs. Lifts a capability Story per cluster, with `cross_links[]` back to the contributing concretes.
5. **Validator gate #2** — same agent as 3a; capability rubric only. Synthesized stories are a *new* artifact, not a derivative, so they re-pass the bar.
6. **Dependency Resolver** — story↔story dependencies, stage-split sibling links, capability↔child links. Finalizes the artifact graph for Jira push.

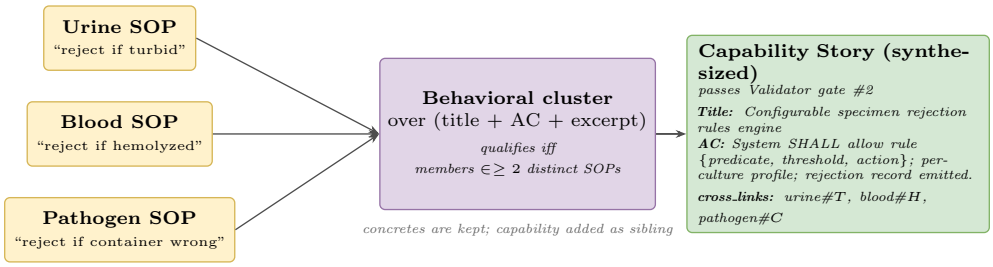
Four story shapes — per-SOP extraction replicates the Cyto mix



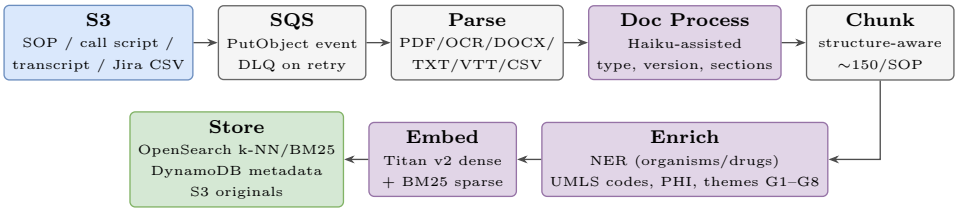
Story schema (shape drives Validator routing):

```
{ id, epic_id, title, description,
  shape ∈ {capability, workflow-stage-split, configuration-instance, cleanup},
  acceptance_criteria[], source_citations[], dependencies[], cross_links[],
  estimated_complexity ∈ {S, M, L}, source_chunks[] }
```

Cross-SOP synthesis — recurrence becomes capability



Ingestion substrate — S3 to indexed chunks



I/O trace — one document through the pipeline (`SOP-MICRO-014_Gram-Stain_v3.docx`):

```
[parse out]
text: 28,420 chars / 18 pages
format: docx
ocr_used: false, parse_ms: 320
```

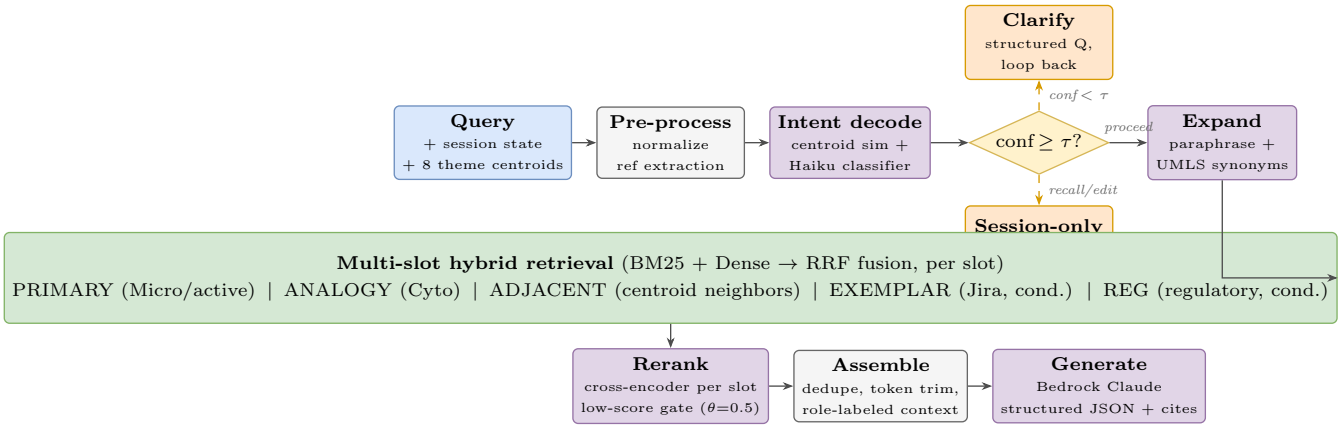
```
[doc process out]
doc_id: doc_micro_014_v3_abc12
discipline: Microbiology, version: v3
section_map: {Purpose:[200,450], Procedure:[721,2400],
AC:[2401,3100], QC:[3101,3650]}
```

```
[enrich dst, chunk_001]
entities: {organisms:["Gram-positive cocci"],
specimens:["sputum"], codes:["87209"]}
phi_detected: false
themes: {G2: 0.84, G5: 0.41}
```

```
[embed dst, chunk_001]
dense_vector: [0.0234, -0.1849, ..., 0.2891] (1024-d, Bedrock Titan v2)
bm25_tokens_enriched: ["slide","preparation","87209","smear","primary","source",...]
```

```
[store, sinks written]
PUT chunk_001 → OpenSearch k-NN index (vector)
PUT chunk_001 → OpenSearch BM25 index (sparse)
PUT chunk_001 → DynamoDB chunk_metadata (entities, themes, phi_status, version)
```

Retrieval substrate — per query, real-time



I/O trace — one query: "Generate epics for specimen rejection in microbiology"

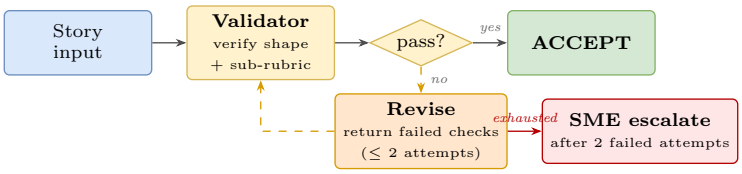
```
[intent decode out]
centroid_dist: {G1: 0.81, G5: 0.18}
classifier: {action: generate, themes: [G1, G5], discipline: Micro, granularity: epic,
refs: [], conf: 0.91}
decision: PROCEED (τ=0.7, 0.91 ≥ τ)
```

```
[retrieve out] 48 candidate chunks
  slot_1 PRIMARY (Micro/G1+G5): 20
  slot_2 ANALOGY (Cyto/G1+G5): 10
  slot_3 ADJACENT (Micro/G2 neighbors): 10
  slot_4 EXEMPLAR (Jira, generate∧epic): 5
  slot_5 REG (CLSI on G5): 3
```

```
[rerank + assemble out]
trim: 8/4/3/3/2 = 20 chunks (all slots ≥ θ=0.5)
prompt sections: PRIMARY, ANALOGY, ADJACENT, EXEMPLAR, REG
+ session drafts + output schema
total tokens: 6,840
```

```
[generate out] structured JSON
{ epic_id: epic_draft_3,
  title: "Specimen Rejection Workflow",
  theme: G1,
  citations: ["s3://.../SOP-MICRO-007 §AC",
              "s3://.../SOP-CYTO-021 §Procedure",
              "CLSI M22 §3.4"],
  stories: ["Validate transport temp", ...] }
```

Validator — shape-aware revise/escalate flow



Sub-rubrics by shape (cheat sheet):

- **Capability:** MUST/SHALL; testable AC; no ambiguous quantifiers; configurability boundaries; SOP citation; complexity S/M/L; edge cases enumerated.
- **Stage-split:** stage explicit in title; entry/exit observable; siblings enumerated and cross-linked; transition contract; boundary cases.
- **Configuration-instance:** typed concrete values; target table/profile named; SOP excerpt verbatim; scope (culture/lab) stated. *MUST/SHALL not required.*
- **Cleanup:** target artifact named (id, screen path, file path); before/after observable; regression risk acknowledged; DoD includes smoke test. *MUST/SHALL not required.*

What ships + cross-cutting

- **Output A — Jira artifacts.** Epics (with optional `cyto_epic_analog`), per-SOP concrete Stories in 4 shapes, synthesized capability Stories cross-linked to children, dependency edges between them.
- **Output B — Per-culture YAML profile.** One per culture in scope; values typed (units / enums / nullable rules); SOP excerpt cited inline; `related_story_ac` links to Story for traceability.

Example YAML excerpt (`cultures/urine.yaml`):

```
centrifugation:
  speed:   {value:  1500, units:  g}   # cite:  SOP-MICRO-014 §3.2 L41--43
  duration:  {value:  5,   units:  min} # cite:  SOP-MICRO-014 §3.2 L44
  temp:    {value:  4,   units:  degC} # cite:  SOP-MICRO-014 §3.2 L45
related_story_ac:  STORY-1042#AC-2
```

Cross-cutting: KMS encryption (rest + transit); IAM least-privilege + VPC endpoints (Bedrock calls stay in-VPC); DLQ for parse/OCR/embed failures; CloudWatch metrics + alarms (e.g. OCR fail-rate > 5%); CloudTrail audit log; idempotent ingestion via content-hash dedupe.

Generalization: when Histology arrives, the same agents and the same retrieval substrate run — only the exemplar corpus and config priors get extended. No model retraining.